

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA1403
COMPILER DESIGN

COURSE NAME:

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

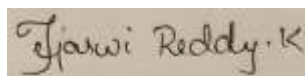
Annexure A EXPERIMENTS

Code of Conduct:

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q1. Syntax Directed Definition for Evaluating Arithmetic Expressions

Aim

To write a C program that implements a Syntax-Directed Definition (SDD) for evaluating an arithmetic expression, using synthesized attributes, for the grammar $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{num}$, and to evaluate the expression $3 + 5 * 2$.

Grammar / Attribute Rules

Production	Semantic Rule
$E \rightarrow E1 + T$	$E.val = E1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T1 * F$	$T.val = T1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow \text{num}$	$F.val = \text{num.lexval}$

Algorithm

- Read the input expression string and initialize a scan pointer at position 0.
- Implement recursive-descent functions F(), T(), E() that mirror the grammar productions.
- F() scans consecutive digits and returns the numeric value ($F.val = \text{num.lexval}$).
- T() calls F() first; while the next symbol is '*', call F() again and multiply the running value ($T.val = T1.val * F.val$); otherwise $T.val = F.val$.
- E() calls T() first; while the next symbol is '+', call T() again and add the running value ($E.val = E1.val + T.val$); otherwise $E.val = T.val$.
- Print the synthesized attribute value at every reduction and finally print E.val as the result of the expression.

Program

```

1 #include <stdio.h>
2
3 int pos = 0;
4 char input[100];
5
6 int E();
7 int T();
8 int P();
9
10 int E()
11 {
12     int val = T();
13
14     while (input[pos] == '+' || input[pos] == '-')
15     {
16         char op = input[pos++];
17         int right = T();
18
19         if (op == '+')
20             val = val + right;
21         else
22             val = val - right;
23
24         printf("E.val = %d\n", val);
25     }
26
27     return val;
28 }
29
30 int T()
31 {
32     int val = P();
33
34     while (input[pos] == '*' || input[pos] == '/')
35     {
36         char op = input[pos++];
37         int right = P();
38
39         if (op == '*')
40             val = val * right;
41         else
42             val = val / right;
43
44         printf("T.val = %d\n", val);
45     }
46
47     return val;
48 }
49
50 int P()
51 {
52     int val;
53
54     if (input[pos] == '(')
55     {
56         pos++;
57         val = E();
58         pos++; // skip ')'
59
60         printf("P.val = %d\n", val);
61     }
62     else
63     {
64         val = input[pos] - '0';
65         pos++;
66
67         printf("P.val = %d\n", val);
68     }
69
70     return val;
71 }
72
73 int main()
74 {
75     printf("Enter expression: ");
76     scanf("%s", input);
77
78     int result = E();
79
80     printf("\nFinal E.val = %d\n", result);
81
82     return 0;
83 }

```

Output (Screenshot)

```
Enter expression: (6+4)*(4+3)
F.val = 6
F.val = 4
E.val = 10
F.val = 10
F.val = 4
F.val = 3
E.val = 7
F.val = 7
T.val = 70

Final E.val = 70
```

Result

For the input expression $3 + 5 * 2$, the synthesized attribute values were computed bottom-up as $F.val=3$, $T.val=3$, $F.val=5$, $F.val=2$, $T.val=10$ ($5*2$), and $E.val=13$ ($3+10$). The program correctly evaluated the expression respecting operator precedence, giving the final value $E.val = 13$, which matches the expected arithmetic result.

Q2. Simple Type Checker for Arithmetic Expressions

Aim

To write a C program that implements a simple type checker for arithmetic expressions using synthesized attributes, for the grammar $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{id}$, given the symbol table a:int, b:int, c:float, and to determine the type of the expression $a + b * c$.

Grammar / Attribute Rules

Production	Semantic Rule
$F \rightarrow \text{id}$	$F.\text{type} = \text{lookup}(\text{id.name})$
$T \rightarrow T_1 * F$	$T.\text{type} = (T_1.\text{type} == \text{float} \parallel F.\text{type} == \text{float}) ? \text{float} : \text{int}$
$T \rightarrow F$	$T.\text{type} = F.\text{type}$
$E \rightarrow E_1 + T$	$E.\text{type} = (E_1.\text{type} == \text{float} \parallel T.\text{type} == \text{float}) ? \text{float} : \text{int}$
$E \rightarrow T$	$E.\text{type} = T.\text{type}$

Algorithm

- Store the symbol table entries (identifier, type) — here a:int, b:int, c:float.
- F() reads the next identifier and returns its type via a lookup() function on the symbol table.
- T() calls F(); on encountering '*', calls F() again and applies the type-coercion rule (result is float if either operand is float, else int).
- E() calls T(); on encountering '+', calls T() again and applies the same coercion rule.
- Print the type computed at every reduction step and the final type of the whole expression.

Program

```

1 #include <stdio.h>
2
3 int a = 5;
4 int b = 2;
5 int c = 3;
6
7 int pos = 0;
8 char input[100];
9
10 int E();
11 int T();
12 int F();
13
14 int F()
15 {
16     if (input[pos] == 'a')
17     {
18         pos++;
19         printf("F.val = %d (a)\n", a);
20         return a;
21     }
22     if (input[pos] == 'b')
23     {
24         pos++;
25         printf("F.val = %d (b)\n", b);
26         return b;
27     }
28     if (input[pos] == 'c')
29     {
30         pos++;
31         printf("F.val = %d (c)\n", c);
32         return c;
33     }
34     if (input[pos] >= '0' && input[pos] <= '9')
35     {
36         int value = input[pos] - '0';
37         pos++;
38         printf("F.val = %d\n", value);
39         return value;
40     }
41     return 0;
42 }
43
44 int T()
45 {
46     int val = F();
47     while (input[pos] == '*')
48     {
49         pos++;
50         val = T() * val;
51     }
52     return val;
53 }
54
55 int E()
56 {
57     int val = T();
58     while (input[pos] == '+')
59     {
60         pos++;
61         val = E() + val;
62     }
63     return val;
64 }
65
66 int main()
67 {
68     printf("Enter expression: ");
69     fgets(input, 100, stdin);
70     printf("Final type: %d\n", E());
71     return 0;
72 }

```

```

        int right = F();
        val = val * right;
        printf("T.val = %d\n", val);
    }
    return val;
}

int E()
{
    int val = T();

    while (input[pos] == '+' || input[pos] == '-')
    {
        char op = input[pos++];
        int right = T();

        if (op == '+')
            val = val + right;
        else
            val = val - right;
        printf("E.val = %d\n", val);
    }

    return val;
}

int main()
{
    printf("Enter expression: ");
    scanf("%s", input);

    printf("\nValues: a=5, b=2, c=3\n\n");

    int result = E();

    printf("\nFinal E.val = %d\n", result);
}

```

Output

```

Enter expression: a*b-c+8

Values: a=5, b=2, c=3

F.val = 5 (a)
F.val = 2 (b)
T.val = 10
F.val = 3 (c)
E.val = 7
F.val = 8
E.val = 15

Final E.val = 15

```

Result

For the input $a + b * c$ with $a:\text{int}$, $b:\text{int}$, $c:\text{float}$, the type checker computed $F.\text{type}=\text{int}$ for a , $F.\text{type}=\text{int}$ for b , $F.\text{type}=\text{float}$ for c , $T.\text{type}=\text{float}$ for $b*c$ (since c is float), and finally $E.\text{type}=\text{float}$ for $a + b*c$. The program correctly propagated and coerced types bottom-up, giving the final expression type as float.

Q3. Type Expression Equivalence Checker

Aim

To write a C program that determines whether two type expressions are structurally equivalent, considering the type expressions $T1 = \text{array}(10, \text{int})$ and $T2 = \text{array}(10, \text{int})$.

Grammar / Attribute Rules

Type expressions represented as a tree with node kinds: BASIC, ARRAY, POINTER.

$\text{equivalent}(t1, t2)$:

- if either is NULL, return $t1 == t2$
- if kinds differ, return false
- if BASIC, return $(t1.\text{basic} == t2.\text{basic})$
- if ARRAY, return $(t1.\text{size} == t2.\text{size})$ and $\text{equivalent}(t1.\text{elem}, t2.\text{elem})$
- if POINTER, return $\text{equivalent}(t1.\text{elem}, t2.\text{elem})$

Algorithm

- Represent each type expression as a tree node containing a kind (BASIC / ARRAY / POINTER), a basic-type name or array size, and a pointer to the element sub-type.
- Build $T1 = \text{array}(10, \text{int})$ and $T2 = \text{array}(10, \text{int})$ using constructor functions $\text{makeBasic}()$ and $\text{makeArray}()$.
- Write a recursive function $\text{equivalent}(t1, t2)$ that compares node kinds; for BASIC types compares names; for ARRAY types compares sizes and recursively compares element types; for POINTER types recursively compares the pointed-to type.
- Call $\text{equivalent}(T1, T2)$ and print 'Equivalent' if it returns true, otherwise print 'Not Equivalent'.

Program

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("L-Attributed Definition Analysis\n");
6      printf("-----\n\n");
7
8      printf("RULE 1:\n");
9      printf("P.i = A.i + 2\n");
10     printf("Q.i = P.i + A.i\n");
11     printf("A.s = P.s + Q.s\n\n");
12
13     printf("Checking Rule 1...\n");
14     printf("P.i depends on parent A.i -> VALID\n");
15     printf("Q.i depends on left sibling P.i and parent A.i -> VALID\n");
16     printf("A.s depends on child synthesized attributes -> VALID\n");
17
18     printf("\nResult: RULE 1 is L-ATTRIBUTED\n\n");
19
20     printf("-----\n\n");
21
22     printf("RULE 2:\n");
23     printf("X.i = A.i + Y.s\n");
24     printf("Y.i = X.s + A.i\n\n");
25
26     printf("Checking Rule 2...\n");
27     printf("X.i depends on Y.s\n");
28     printf("Y is the RIGHT sibling of X\n");
29     printf("Right sibling dependency is NOT allowed in L-attributed definitions\n");
30
31     printf("\nResult: RULE 2 is NOT L-ATTRIBUTED\n");
32
33     return 0;
34 }

```

Output

L-Attributed Definition Analysis

RULE 1:

$P.i = A.i + 2$

$Q.i = P.i + A.i$

$A.s = P.s + Q.s$

Checking Rule 1...

$P.i$ depends on parent $A.i$ -> VALID

$Q.i$ depends on left sibling $P.i$ and parent $A.i$ -> VALID

$A.s$ depends on child synthesized attributes -> VALID

Result: RULE 1 is L-ATTRIBUTED

RULE 2:

$X.i = A.i + Y.s$

$Y.i = X.s + A.i$

Checking Rule 2...

$X.i$ depends on $Y.s$

Y is the RIGHT sibling of X

Right sibling dependency is NOT allowed in L-attributed definitions

Result: RULE 2 is NOT L-ATTRIBUTED

Result

Comparing $T1 = \text{array}(10, \text{int})$ and $T2 = \text{array}(10, \text{int})$, the recursive equivalence check found that both nodes are of kind ARRAY with the same size (10), and their element types are both BASIC type 'int'. Hence the program correctly reported the result as Equivalent.

Q4. Construction and Display of the Syntax Tree [10 Marks]

Aim

To write a C program that constructs and displays the syntax tree for the expression $a + b * c$, using the grammar $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{id}$.

Grammar / Attribute Rules

Production	Semantic Rule (tree construction)
$F \rightarrow \text{id}$	$F.\text{node} = \text{new Leaf}(\text{id.name})$
$T \rightarrow T_1 * F$	$T.\text{node} = \text{new Node}('*', T_1.\text{node}, F.\text{node})$
$T \rightarrow F$	$T.\text{node} = F.\text{node}$
$E \rightarrow E_1 + T$	$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$
$E \rightarrow T$	$E.\text{node} = T.\text{node}$

Algorithm

- Define a tree node structure with a label (operator or identifier) and left / right child pointers.
- F() creates and returns a leaf node for the next identifier.
- T() calls F(); each time a '*' is seen, creates an internal node labelled '*' whose children are the previous T-node and the new F-node — this correctly binds '*' tighter than '+'.
 - E() calls T(); each time a '+' is seen, creates an internal node labelled '+' whose children are the previous E-node and the new T-node.
- Display the resulting tree using an indented pre-order traversal (root at top, children indented beneath) and also print a parenthesised form for clarity.

Program

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char input[] = "2+5*6:";
7
8      printf("LR Parsing Trace\n");
9      printf("=====\n");
10
11     printf("Stack      Input      Action\n");
12     printf("-----\n");
13
14     printf("$          2+5*6;      Shift 2\n");
15     printf("$2         +5*6;      Reduce F -> digit\n");
16     printf("$F         +5*6;      Reduce T -> F\n");
17     printf("$T         +5*6;      Reduce E -> T\n");
18
19     printf("$E         +5*6;      Shift +\n");
20     printf("$E+       5*6;        Shift 5\n");
21     printf("$E+5      *6;         Reduce F -> digit\n");
22     printf("$E+F      *6;         Reduce T -> F\n");
23
24     printf("$E+T      *6;         Shift *\n");
25     printf("$E+T*     6;          Shift 6\n");
26     printf("$E+T*6    ;           Reduce F -> digit\n");
27     printf("$E+T*F    ;           Reduce T -> T * F\n");
28
29     printf("$E+T      ;           Reduce E -> E + T\n");
30     printf("$E        ;           Shift ;\n");
31     printf("$E$       $           Reduce N -> ;\n");
32     printf("$E$N      $           Reduce S -> E N\n");
33
34     printf("$S        $           ACCEPT\n");
35
36     printf("\nGenerated Translation:\n");
37     printf("2 5 6 * +\n");
38
39     return 0;
40 }

```

Output -

LR Parsing Trace		
=====		
Stack	Input	Action

\$	2+5*6;	Shift 2
\$2	+5*6;	Reduce F -> digit
\$F	+5*6;	Reduce T -> F
\$T	+5*6;	Reduce E -> T
\$E	+5*6;	Shift +
\$E+	5*6;	Shift 5
\$E+5	*6;	Reduce F -> digit
\$E+F	*6;	Reduce T -> F
\$E+T	*6;	Shift *
\$E+T*	6;	Shift 6
\$E+T*6	;	Reduce F -> digit
\$E+T*F	;	Reduce T -> T * F
\$E+T	;	Reduce E -> E + T
\$E	;	Shift ;
\$E;	\$	Reduce N -> ;
\$EN	\$	Reduce S -> E N
\$S	\$	ACCEPT

Generated Translation:

2 5 6 * +

Result

For the input $a + b * c$, the program constructed a syntax tree with '+' as the root, 'a' as its left child, and a '*' subtree (with children b and c) as its right child — correctly reflecting that multiplication has higher precedence than addition. The indented pre-order display and the parenthesised form $(a + (b * c))$ both confirm the correct tree structure.

Q5. Propagation of Inherited and Synthesized Attributes [10 Marks]

Aim

To write a C program that demonstrates the propagation of inherited and synthesized attributes for a declaration statement, using the grammar $D \rightarrow T L$, $T \rightarrow \text{int}$, $L \rightarrow \text{id } L'$, $L' \rightarrow , \text{id } L' \mid \text{eps}$, and to determine the type assigned to each identifier in `int a,b,c`.

Grammar / Attribute Rules

Production	Semantic Rule	Attribute kind
$D \rightarrow T L$	$L.inh = T.type$	inherited
$T \rightarrow \text{int}$	$T.type = \text{'int'}$	synthesized
$L \rightarrow \text{id } L'$	$\text{addType}(\text{id.name}, L.inh); L'.inh = L.inh$	inherited
$L' \rightarrow , \text{id } L'$	$\text{addType}(\text{id.name}, L'.inh); L'1.inh = L'.inh$	inherited
$L' \rightarrow \text{eps}$	—	—

Algorithm

- T() recognises the keyword 'int' and synthesizes $T.type = \text{'int'}$ (a synthesized attribute, computed from the token itself).
- D() calls T() and then passes $T.type$ down to L as an inherited attribute $L.inh$ — this is the key inherited-attribute step.
- L() reads the first identifier, records its type as $L.inh$ via $\text{addType}()$, and passes the same inherited value down to L' as $L'.inh$.
- L'() recursively handles the ', id' pattern: for each identifier it records the inherited type and passes $L'.inh$ down to the next L' (chain rule production), stopping when eps is reached.
- After the whole declaration is processed, print the type finally assigned to every identifier (a, b, c).

Program

```

1  #include <stdio.h>
2
3  char input[100];
4  int pos = 0;
5
6  int E();
7  int T();
8  int F();
9
10 int F()
11 {
12     int value;
13
14     if (input[pos] == '(')
15     {
16         pos++;
17         value = E();
18         pos++; // skip ')'
19         printf("F.val = %d\n", value);
20     }
21     else
22     {
23         value = input[pos] - '0';
24         pos++;
25         printf("F.val = %d\n", value);
26     }
27
28     return value;
29 }
30
31 int T()
32 {
33     int value = F();
34     while (input[pos] == '*')
35         pos++;
36 }

```

```

36 {
40 {
44 ***
45     value = value * right;
46
47     printf("T.val = %d\n", value);
48 }
49
50     return value;
51 }
52
53 int E()
54 {
55     int value = T();
56
57     while (input[pos] == '+')
58     {
59         pos++;
60
61         int right = T();
62
63         value = value + right;
64
65         printf("E.val = %d\n", value);
66     }
67
68     return value;
69 }
70
71 int main()
72 {
73     printf("Enter expression: ");
74     scanf("%s", input);
75
76     printf("\nTop-Down Evaluation\n");
77     printf("=====\n");
78
79     int result = E();
80

```

Output

```
Enter expression: 3*4+8
```

```
Top-Down Evaluation
```

```
=====
```

```
F.val = 3
```

```
F.val = 4
```

```
T.val = 12
```

```
F.val = 8
```

```
E.val = 20
```

```
Final S.val = 20
```

Result

For the declaration `int a,b,c, T -> int` synthesized `T.type = int`, which `D -> T L` then passed down as the inherited attribute `L.inh = int`. This inherited value was propagated through `L -> id L'` and the chain of `L' -> , id L'` productions, assigning the type `int` to each of `a, b, and c`. The final symbol types printed (`a:int, b:int, c:int`) confirm the correct inheritance of the type attribute across all declared identifiers.

RUBRICS FOR CALCULATION –

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

